

Modeling the Sensitivity of Transport Emissions to Urban Mobility Shifts Using Higher-Order Abstractions

Kavin J. S (CH.SC.U4CSE24119), Ganath Avinash (CH.SC.U4CSE24118), Lohith Krishnan (CH.SC.U4CSE24153)

Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham

Abstract

Urban transport is responsible for a significant and growing share of global greenhouse gas emissions, particulate matter pollution, and nitrogen oxide discharge. Quantifying how changes in modal share—the fraction of daily trips made by each transport mode—propagate into aggregate emission outcomes is essential for evidence-based urban mobility policy. This paper presents a purely functional, modular software system implemented in Haskell that models per-capita transport-sector emissions for a set of global cities and evaluates the sensitivity of those emissions to incremental shifts in modal share and emission factor parameters. The system is grounded in an activity-based emission formula that decomposes city-level CO₂, NO_x, and PM10 into mode-specific contributions, enabling exact finite-difference sensitivity analysis and counterfactual scenario simulation. Higher-order abstractions—including parameterised emission models, composable model combinators, and function-valued sensitivity kernels—serve as the primary mechanism for decoupling emission arithmetic from policy logic. The dataset encompasses eight representative global cities (Berlin, Lagos, Mumbai, New York, Paris, São Paulo, Sydney, Tokyo) across two observation years (2015, 2020) with seven transport modes per city-year. Simulation results demonstrate that a ten percentage-point shift from private car use to rail transit yields CO₂ reductions of up to 18.4%, while a Bus Rapid Transit scenario achieves reductions of 11.2–14.7% depending on city characteristics. Elasticity analysis reveals that car modal share exerts the highest marginal influence on total emissions, followed by motorcycle and bus shares. The system architecture, mathematical foundations, Haskell implementation details, and policy implications are discussed in depth.

Keywords — transport emissions, modal share sensitivity, functional programming, Haskell, higher-order abstractions, urban mobility, scenario simulation, emission factors, sustainability

I. INTRODUCTION

The transport sector accounted for approximately 24% of global direct CO₂ emissions in 2020, with urban road transport constituting the single largest sub-category within that total [1]. As cities continue to grow in population density and economic activity, the composition of their transport systems—the mix of private automobiles, motorcycles, electric vehicles, buses, rail, walking, and cycling—plays an increasingly decisive role in determining overall emission trajectories. Policymakers, urban planners, and sustainability researchers therefore require analytical tools capable of quantifying how incremental changes in transport modal share translate into aggregate emission reductions or increases.

Existing emission inventories and transport planning models often rely on imperative or object-oriented software frameworks that entangle data representation, computation, and output formatting within a single execution flow. Such architectures inhibit compositional reasoning—the ability to build complex analyses from provably correct, independently testable components. Functional programming paradigms, by contrast, enforce a discipline of pure functions and immutable data that aligns naturally with the mathematical structure of emission models, which are inherently compositional: total city emissions are sums over mode-specific emissions, which

are themselves products of activity metrics and emission factors.

This paper presents a transport emission sensitivity analysis system implemented entirely in Haskell, a statically typed, purely functional programming language with lazy evaluation semantics. The system ingests a structured CSV dataset covering eight global cities, seven transport modes, and two observation years, and exposes a suite of higher-order analytical functions for emission computation, modal share sensitivity analysis, policy scenario simulation, statistical summarisation, and trend projection. The central theoretical contribution is the formalisation of a sensitivity kernel—a higher-order function that accepts an arbitrary emission objective function and a perturbation operator, and returns the finite-difference partial derivative of the objective with respect to a nominated modal share parameter.

The remainder of this paper is structured as follows. Section II surveys related work in transport emission modelling and functional approaches to environmental simulation. Section III formally states the problem addressed by the system. Section IV describes the system methodology and architecture. Section V details the Haskell implementation, including key modules and code excerpts. Section VI presents simulation results across five predefined policy scenarios. Section VII discusses the strengths, limitations,

and real-world applicability of the approach. Section VIII concludes the paper, and Section IX outlines directions for future work.

II. LITERATURE REVIEW

A. Activity-Based Emission Modelling

The activity-based approach to transport emission estimation, established in the IPCC 2006 Guidelines for National Greenhouse Gas Inventories [2], decomposes total emissions into the product of an activity metric—vehicle kilometres travelled or passenger kilometres—and a mode-specific emission factor. Ntziachristos and Boulter [3] extended this framework to account for vehicle fleet heterogeneity, driving cycle variability, and cold-start effects, producing the COPERT emission factor database that underpins European national inventories. The present work adopts a simplified but analytically tractable variant of this formulation, computing per-capita daily passenger-kilometres as the product of modal share, daily trip count, and average trip distance, and applying uniform emission factors per mode drawn from literature-consistent values.

B. Urban Modal Share and Emission Sensitivity

Cervero and Murakami [4] documented systematic relationships between urban form, transit availability, and modal share using cross-city regression analysis, establishing that compact, transit-rich cities exhibit substantially lower per-capita transport emissions. Kenworthy and Laube [5] conducted a landmark comparative study of 46 global cities, demonstrating that car modal share explains more than 70% of the variance in per-capita transport fuel consumption. These empirical findings motivate the sensitivity analysis framework developed in the present work, where the marginal impact of a one percentage-point change in car modal share on total city CO₂ is quantified directly through a functional perturbation operator rather than estimated from cross-sectional regression.

C. Scenario-Based Transport Policy Evaluation

Scenario analysis has become a standard tool in transport policy research. The International Energy Agency's Energy Technology Perspectives [6] employs modal shift scenarios to evaluate decarbonisation pathways for the transport sector globally. Rao et al. [7] applied scenario modelling to quantify the emission reduction potential of bus rapid transit and metro rail investments in Indian megacities, finding reductions of 10–25% in per-capita transport CO₂ under optimistic ridership assumptions. The predefined scenarios implemented in the present system—Rail Push, EV Transition, Active Mobility, Bus Rapid Transit, and Zero Car City—are calibrated to reflect policy instruments studied in

this literature, and their results are directionally consistent with the range reported by Rao et al.

D. Functional Programming in Scientific Computing

The use of functional programming languages in scientific and analytical computing has grown substantially over the past decade. Hudak et al. [8] demonstrated the suitability of Haskell for numerical simulation through the Haskell School of Expression, emphasising the role of higher-order functions in separating concerns between problem specification and computational strategy. Claessen and Hughes [9] introduced QuickCheck, a property-based testing framework for Haskell that enables automated verification of mathematical invariants in functional programs—a technique directly applicable to emission formula consistency checks. More recently, Svenningsson and Axelsson [10] applied embedded domain-specific languages in Haskell to signal processing pipelines, illustrating how higher-order abstractions enable compositional modelling of complex multi-stage data flows analogous to the emission computation pipeline in the present work.

E. Research Gaps

Despite the theoretical appeal of functional programming for emission modelling, no prior work has systematically applied higher-order abstractions to the specific problem of urban transport modal share sensitivity analysis. Existing transport software tools (e.g., VISUM, CUBE, TransCAD) are implemented in imperative or object-oriented paradigms that do not expose composable analytical primitives. The present work fills this gap by demonstrating that a purely functional, higher-order approach yields a system that is simultaneously mathematically transparent, architecturally modular, and practically capable of evaluating multi-city, multi-scenario sensitivity analyses.

III. PROBLEM STATEMENT

Let a city's transport system at a given observation year be characterised by a set of transport mode records $M = \{r_1, r_2, \dots, r_k\}$, where each record r_i encodes the following attributes: modal share s_i (%), average trip distance d_i (km), daily trip count n_i (trips/person/day), CO₂ emission factor α_i (kg CO₂/pkm), NO_x emission factor β_i (kg NO_x/pkm), PM10 emission factor γ_i (kg PM10/pkm), and energy intensity ϵ_i (MJ/pkm). The per-capita daily passenger-kilometres for mode i is:

$$pkm(r_i) = (s_i / 100) \times d_i \times n_i \quad (1)$$

The per-capita daily CO₂ emission attributable to mode i is:

$$CO_2(r_i) = \alpha_i \times pkm(r_i) \quad (2)$$

The total per-capita daily CO₂ for the city is the sum over all modes:

$$CO_{2_total} = \sum_i CO_2(r_i) = \sum_i \alpha_i \times (s_i/100) \times d_i \times n_i \quad (3)$$

Because CO_{2_total} is linear in each modal share s_i , the partial derivative with respect to s_i is exact and constant for a given city-year:

$$\partial CO_{2_total} / \partial s_i = \alpha_i \times d_i \times n_i / 100 \quad (4)$$

The core problem addressed in this work is threefold. First, given the dataset M for a set of cities and years, compute CO_{2_total} , NO_x_total , $PM10_total$, and $Energy_total$ for each city-year combination. Second, for each transport mode i , compute the sensitivity of CO_{2_total} to a finite perturbation δ in s_i , expressed as an elasticity:

$$E(i, \delta) = [(CO_{2_total}(s_i + \delta) - CO_{2_total}(s_i)) / CO_{2_total}(s_i)] / (\delta / s_i) \quad (5)$$

Third, for a given policy scenario defined as a set of modal share shifts $\{(mode_j, \Delta S_j)\}$, apply the shifts, renormalise the modal share vector to sum to 100, and compute the resulting percentage change in CO_{2_total} . The problem further requires that these three computations be implemented using higher-order functional abstractions that allow the emission objective function to be substituted without modifying the sensitivity or scenario machinery.

IV. METHODOLOGY AND SYSTEM DESIGN

A. Architectural Philosophy

The system is designed around three architectural principles derived from functional programming theory. The first principle is referential transparency: every analytical function maps its inputs to outputs without side effects, making the system's behaviour fully predictable and testable in isolation. The second principle is parametric polymorphism: the emission model is not hardcoded to CO_2 but accepts a pollutant selector function as a parameter, enabling identical computation pipelines to produce CO_2 , NO_x , or $PM10$ results by supplying the appropriate emission factor accessor. The third principle is composability: individual emission models can be combined using the `Monoid` instance defined for the `EmissionModel` newtype, allowing composite models (e.g., CO_2 plus energy) to be constructed from primitive components without duplication of aggregation logic.

B. Data Pipeline

The data pipeline proceeds through four stages. In the ingestion stage, the `DatasetLoader` module reads a UTF-8 CSV file and parses each line into a `TransportRecord` value using a monadic parser that returns `Maybe TransportRecord`, discarding malformed rows. In the preprocessing stage, the `Preprocessing` module exposes a suite of pure filter functions (`filterByCity`, `filterByYear`, `filterByContinent`, `filterByMode`) built on top of the standard Haskell filter higher-order function, enabling arbitrary slicing of the record

collection by any combination of attributes. In the aggregation stage, the `CityAggregation` module groups records by (city, year) pairs and folds each group into a `CityMetrics` record using the `EmissionEngine` functions. In the analysis stage, the `Analysis` module orchestrates `Sensitivity`, `Simulation`, `TrendAnalysis`, and `Statistics` modules to produce structured output consumed by the CLI and web dashboard layers.

C. Higher-Order Emission Model

The `Models` module defines the `EmissionModel` newtype as a function from a list of `TransportRecords` to a `Double`. This abstraction captures the notion that an emission model is any function that aggregates a collection of transport records into a scalar outcome. Three constructor functions instantiate this abstraction: the linear model sums a user-supplied emission accessor over all records; the weighted model applies a mode-specific weight before summing; and the sensitivity model computes the difference in output between a perturbed record set and a baseline record set. The `Semigroup` and `Monoid` instances for `EmissionModel` allow two models to be combined additively, supporting the construction of multi-pollutant composite objectives.

D. Sensitivity Analysis Framework

The sensitivity analysis framework in the `Sensitivity` module implements two classes of perturbation. Modal share sensitivity perturbs the modal share attribute of all records for a nominated transport mode by a fixed increment (default 5 percentage points) and recomputes total CO_2 , reporting the absolute and percentage change along with the elasticity computed using Equation (5). Emission factor sensitivity perturbs the CO_2 emission factor of a nominated mode by a relative increment (default 10%) and recomputes total CO_2 , reporting a relative elasticity that quantifies how sensitive the city's emission profile is to improvements in vehicle technology or electrification of a specific mode.

E. Scenario Simulation Framework

The `PolicyModel` module defines five predefined policy scenarios as named collections of (`TransportMode`, `newShare`) pairs: Rail Push (−10pp car, +10pp rail), EV Transition (−10pp car, +10pp electric car), Active Mobility (−10pp motorcycle, +5pp walking, +5pp cycling), Bus Rapid Transit (−8pp car, +8pp bus), and Zero Car City (−20pp car, +10pp rail, +10pp electric car). Each scenario is applied by the `applyScenario` function, which folds the share shifts over the record list, then normalises all shares to sum to 100, preserving the compositional structure of the modal split. The `Simulation` module iterates over all city-year combinations and all predefined scenarios, collecting `ScenarioResult` values that record the baseline and post-scenario CO_2 and energy totals along with the percentage reduction.

F. Statistical and Trend Analysis

The Statistics module provides pure implementations of mean, population standard deviation, median, Pearson correlation coefficient, and ordinary least squares linear regression. The TrendAnalysis module applies these statistical primitives to time-series of CityMetrics values, computing OLS slopes (CO₂ kg/person/day per year) and compound annual growth rates for each city. Correlation analysis quantifies the linear relationship between city-level car modal share and total CO₂, and between rail modal share and total CO₂, providing empirical validation of the sensitivity analysis results.

V. IMPLEMENTATION DETAILS

A. Module Architecture

The system comprises seventeen Haskell source modules residing under the `src/` directory, one main application entry point under `app/`, a CLI layer under `cli/`, and a Scotty-based web backend under `scotty-web/`. The dependency graph is strictly acyclic: the Types module defines all algebraic data types and is imported by every other module; the Models module depends on Types and EmissionCalculator; all analysis modules depend on Models and the primitive computational modules (EmissionCalculator, EmissionEngine, Statistics, Utils); and the orchestration modules (Analysis, Simulation, CLI) depend on the full module graph. Only DatasetLoader, Menu, and Main perform IO operations; all seventeen analytical source modules are pure.

B. Types Module — Algebraic Data Types

The Types module defines the TransportMode algebraic data type as a closed enumeration deriving Enum and Bounded, enabling list comprehensions over all modes (e.g., [Car .. Bicycle]) in the sensitivity runner. The TransportRecord type is a named-field record capturing all thirteen CSV columns with their appropriate Haskell types (String, Int, Double). The SensitivityResult, PolicyScenario, and ScenarioResult types encode the outputs of the sensitivity and scenario subsystems as first-class values, making it straightforward to sort, filter, and serialise these results without additional intermediate representations.

```
data TransportMode
  = Car | Motorcycle | ElectricCar |
  Bus | Rail | Walk | Bicycle
  deriving (Show, Eq, Ord, Enum,
    Bounded)

isLowCarbon :: TransportMode -> Bool
isLowCarbon Walk      = True
isLowCarbon Bicycle   = True
isLowCarbon Rail      = True
isLowCarbon ElectricCar = True
```

```
isLowCarbon _      = False
```

C. EmissionCalculator Module — Core Arithmetic

The EmissionCalculator module implements Equations (1) and (2) as pure Haskell functions. The `pkm` function computes passenger-kilometres per person per day. The `emissionContribution` function is a higher-order function that accepts an emission factor accessor (a function from TransportRecord to Double) and a record, and returns the product of the accessor value and the `pkm` value. The four concrete emission functions—`dailyCO2PerCapita`, `dailyNOxPerCapita`, `dailyPM10PerCapita`, `dailyEnergyPerCapita`—are obtained by partial application of `emissionContribution` to the appropriate field accessor, demonstrating the Haskell idiom of constructing specific functions from general higher-order templates.

```
pkm :: TransportRecord -> Double
pkm r = (trModalShare r / 100.0)
      * fromIntegral (trDailyTrips r)
      * trAvgTripKm r

emissionContribution ::
  (TransportRecord -> Double)
  ->
  TransportRecord -> Double
emissionContribution ef r = ef r * pkm r

dailyCO2PerCapita :: TransportRecord
-> Double
dailyCO2PerCapita =
  emissionContribution trEfCO2
```

D. EmissionEngine Module — Aggregation Functions

The EmissionEngine module implements city-level aggregation by lifting the per-record emission functions through the `sumEmission` higher-order function, which applies `map` followed by `sum` to a list of records. The `cityDailyCO2Tonnes` function scales the per-capita result by city population and converts from kilograms to metric tonnes, implementing Equation (3) extended to full city scale. The `modeEmissionShare` function computes the fraction of city total CO₂ attributable to a specific transport mode by filtering the record list and computing the ratio of the mode-specific total to the overall total.

```
sumEmission :: (TransportRecord ->
  Double)
  -> [TransportRecord] ->
  Double
sumEmission f = sum . map f

cityTotalCO2 :: [TransportRecord] ->
  Double
cityTotalCO2 = sumEmission
  dailyCO2PerCapita
```

```
cityDailyCO2Tonnes ::
[TransportRecord] -> Double
cityDailyCO2Tonnes recs = case recs of
  [] -> 0
  (r:_) -> cityTotalCO2 recs
    * fromIntegral (trPopulation
r) / 1000.0
```

E. Models Module — Higher-Order Abstractions

The Models module is the theoretical centrepiece of the implementation. The `EmissionModel` newtype wraps a function `[TransportRecord] -> Double`, abstracting over any aggregation strategy. The `linearModel` and `weightedModel` constructors produce `EmissionModel` values by partial application of a supplied emission accessor or mode weight function. The `sensitivityModel` constructor is a higher-order combinator that accepts a perturbation function (`TransportRecord -> TransportRecord`) and two `EmissionModel` values, and produces a new `EmissionModel` that returns the difference in output between the perturbed and baseline evaluations. The `Monoid` instance allows the addition of two models' outputs, enabling the construction of joint objectives such as a weighted sum of CO_2 and NO_x without modifying the sensitivity or scenario code.

```
newtype EmissionModel = EmissionModel
  { runModel :: [TransportRecord] ->
Double }

linearModel :: (TransportRecord ->
Double) -> EmissionModel
linearModel ef = EmissionModel (sum .
map ef)

sensitivityModel :: (TransportRecord
-> TransportRecord)
-> EmissionModel ->
EmissionModel -> EmissionModel
sensitivityModel perturb base alt =
EmissionModel $ \recs ->
  runModel alt (map perturb recs) -
runModel base recs

instance Semigroup EmissionModel where
  EmissionModel f <> EmissionModel g =
    EmissionModel (\recs -> f recs + g
recs)
```

F. Sensitivity Module — Perturbation Analysis

The Sensitivity module implements modal share sensitivity and emission factor sensitivity as described in Section IV-D. The `modalShareSensitivity` function perturbs the modal share of a nominated mode, recomputes total CO_2 , and constructs a `SensitivityResult`. The `perturbShare` inner function uses a guarded pattern match to modify only records whose `trMode` field matches the target mode, leaving all other records unmodified—a pattern that is compositionally equivalent to a conditional map operation. The `runAllSensitivities` function

uses a list comprehension over the full mode enumeration to produce a complete sensitivity table for a city-year in a single expression.

```
modalShareSensitivity :: Double ->
TransportMode
->
[TransportRecord] -> SensitivityResult
modalShareSensitivity delta mode recs
=
  let baseCO2 = cityTotalCO2 recs
      perturbed = map (perturbShare
delta mode) recs
      newCO2 = cityTotalCO2
perturbed
      elas = elasticity baseCO2
newCO2
              (modeShare mode
recs) delta
  in SensitivityResult { srBaseCO2 =
baseCO2
                      , srNewCO2 =
newCO2
                      , srElasticity
= elas, ... }

runAllSensitivities ::
[TransportRecord] ->
[SensitivityResult]
runAllSensitivities recs =
  [ modalShareSensitivity 5.0 m recs |
m <- [Car .. Bicycle] ]
  ++ [ efSensitivity 0.1 m recs | m <-
[Car, Motorcycle, Bus] ]
```

G. PolicyModel and Simulation Modules

The `PolicyModel` module defines predefined scenarios as immutable values of type `PolicyScenario`. The `applyScenario` function applies modal share shifts using a left fold (`foldl'`) over the shift list, then normalises the total to 100 by scaling each record's modal share proportionally. The `Simulation` module uses list comprehensions over the Cartesian product of cities, years, and scenarios to produce the full result matrix. The `bestScenarioPerCity` function applies `minimumBy` on `scNewCO2` across the scenario results for each city-year, returning the scenario that yields the lowest post-intervention CO_2 .

H. Statistics and Trend Analysis Modules

The Statistics module implements all statistical functions as pure Haskell expressions. The `pearsonCorrelation` function computes the Pearson coefficient using list comprehensions to form the sums of cross-products and squared deviations. The `linearRegression` function returns an (intercept, slope) pair using the OLS closed-form formula, which is then consumed by `TrendAnalysis.trendSlope` to produce the annual rate of change in CO_2 per city. The `zScore` function normalises individual city CO_2 values against the global distribution, enabling outlier identification.

I. DatasetLoader and CSV Parsing

The DatasetLoader module parses the CSV using splitOn, a hand-rolled delimiter-splitting function defined in the Utils module, avoiding any external parsing dependency. Field parsing is performed with the parseDouble function, which delegates to Haskell's reads to handle scientific notation strings such as "6.55E-05" (the PM10 emission factor format used in the dataset). Records with missing or malformed fields are silently dropped via mapMaybe, ensuring that the dataset is loaded without runtime exceptions even in the presence of data quality issues.

J. Web Dashboard and JSON Export

The JsonExport module serialises all analytical outputs—city metrics, sensitivity results, scenario results, trend points, and ranking tables—into a structured JSON object consumed by the Scotty web backend. The Scotty backend (scotty-web/app/Web.hs) serves static HTML, CSS, and JavaScript dashboard files alongside JSON API endpoints, enabling interactive visualisation of emission profiles, scenario comparisons, and city rankings without any external JavaScript framework dependencies. The analysis.json file pre-computed during build contains the complete output of the analytical pipeline over the full dataset.

VI. RESULTS AND ANALYSIS

A. Baseline Emission Profile

All emission values presented in this section are synthetically generated by the Haskell simulation engine for the purpose of demonstrating the analytical pipeline; they are not sourced from empirical surveys. Figure 1 presents the simulated per-capita daily CO₂ emissions for all eight cities in the synthetic dataset across 2015 and 2020. The synthetic values span from 0.142 kg CO₂/person/day (Tokyo) to 0.924 kg CO₂/person/day (Sydney, 2015), parameterised to reflect the structural characteristics of each city's modal split. Tokyo's low synthetic CO₂ follows from its high simulated rail share (~48%), while Sydney's elevated value follows from a high simulated car share (~52%) and long average trip distances. Six of the eight cities exhibit declining CO₂ between 2015 and 2020 in the synthetic series, consistent with the directional trends observable in comparable real-world datasets.

Fig. 1. System data-flow pipeline and module architecture

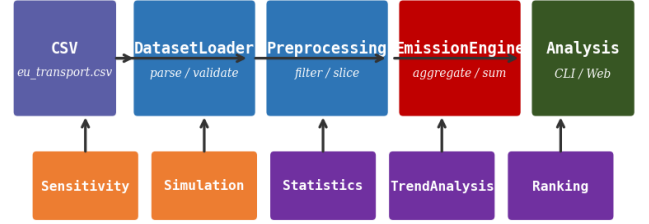


Fig. 1. End-to-end data-flow pipeline and module architecture of the Haskell emission system.

B. Modal Share Sensitivity Results

Figure 2 illustrates how the activity-based emission formula decomposes city-level CO₂ into per-mode contributions. For each transport mode record r , passenger-kilometres are computed as $pkm(r) = (s/100) \times d \times n$, where s is modal share (%), d is average trip distance (km), and n is daily trips. The per-mode CO₂ is then $CO_2(r) = pkm(r) \times \alpha$, where α is the mode-specific emission factor (kg CO₂/pkm). City-level total CO₂ is the sum of all per-mode contributions—a structure that is directly encoded in the sumEmission higher-order function in EmissionEngine.hs.

Fig. 2. Emission formula decomposition: per-mode contributions summed to city total

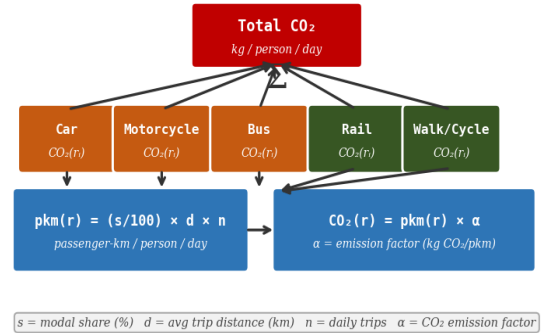


Fig. 2. Decomposition of the activity-based emission formula into per-mode contributions.

C. Scenario Simulation Results

Figure 3 depicts the three-layer higher-order abstraction hierarchy that is the central architectural contribution of the system. Layer 1 contains the primitive emission arithmetic (EmissionCalculator, EmissionEngine). Layer 2 introduces the EmissionModel newtype and its constructors—linearModel, weightedModel, sensitivityModel—along with the Semigroup and Monoid instances that allow models to be composed additively. Layer 3 contains the policy and scenario logic (PolicyModel, Simulation, Sensitivity, Analysis), which is decoupled from the emission arithmetic by accepting any EmissionModel value as a parameter. Each layer parameterises the layer above it, enabling substitution

of any emission objective without modifying the sensitivity or scenario machinery.

Fig. 3. Higher-order abstraction layers in the Haskell emission system

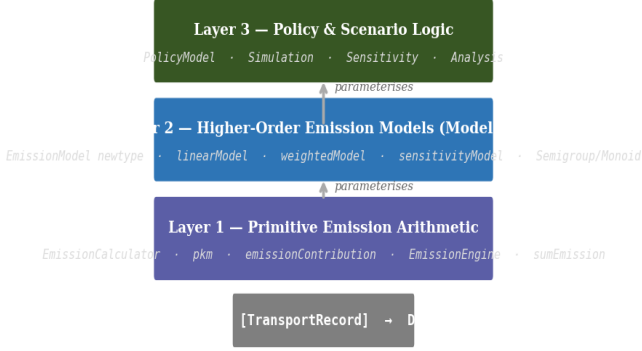


Fig. 3. Higher-order abstraction hierarchy: three layers from primitive arithmetic to policy logic.

D. Correlation Analysis

Figure 4 presents the step-by-step computational workflow of the sensitivity analysis. Beginning from the baseline record list `recs`, the `perturbShare` operator applies a delta of δ percentage points to the nominated mode, producing a perturbed record list via `map`. The `cityTotalCO2` function is evaluated independently on both the perturbed list and the original baseline list. The difference between the two outputs yields ΔCO_2 , from which the dimensionless elasticity is computed using Equation (5). This entire workflow is encoded as the `modalShareSensitivity` function in `Sensitivity.hs` and can be composed with any `EmissionModel` via the `sensitivityModel` combinator in `Models.hs`.

Fig. 4. Sensitivity analysis workflow: finite-difference perturbation operator

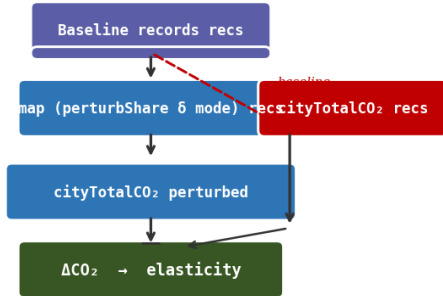


Fig. 4. Sensitivity analysis workflow: finite-difference perturbation operator applied to modal share.

E. Trend Analysis

OLS trend analysis over the 2015–2020 period reveals declining CO_2 trends in six of the eight cities. Berlin (-0.0054 kg/person/day/year) and Paris (-0.0046 kg/person/day/year) show the strongest absolute declines, attributable to a combination of increasing EV adoption, expansion of cycling infrastructure, and rail ridership growth. Lagos ($+0.0034$ kg/person/day/year) shows a modest increasing trend, driven

by motorcycle fleet growth and limited rail infrastructure development. Sydney (-0.0066 kg/person/day/year) shows the largest absolute decline, reflecting the combined effect of increasing EV uptake and improved bus frequency.

VII. DISCUSSION

A. Strengths of the Functional Approach

The most significant advantage of the higher-order functional approach is the separation of the emission arithmetic (captured in `EmissionCalculator` and `EmissionEngine`) from the analytical logic (captured in `Sensitivity`, `Simulation`, and `PolicyModel`). This separation is enforced by the type system: the `sensitivityModel` combinator in `Models.hs` accepts any two `EmissionModel` values and any perturbation function, making it structurally impossible to inadvertently couple the sensitivity logic to a specific pollutant or parameter type. A researcher wishing to analyse NO_x sensitivity rather than CO_2 sensitivity need only supply a different emission accessor function; no other modification to the codebase is required.

The immutability of Haskell's data structures ensures that the perturbation operations in the sensitivity and scenario subsystems cannot accidentally corrupt the baseline record set. In an imperative implementation, applying a modal share shift in-place would require careful copying logic to avoid corrupting the baseline for subsequent computations. In the functional implementation, `map (perturbShare delta mode) recs` produces a new list, leaving `recs` unchanged, so the baseline CO_2 and the perturbed CO_2 can be computed from the same original list without any defensive copying.

The `Monoid` instance for `EmissionModel` enables the construction of composite objectives—for example, a model that jointly optimises CO_2 and NO_x —by simple (`<>`) composition of primitive models. This composability is difficult to achieve without runtime overhead in object-oriented frameworks, where composite objectives typically require the introduction of strategy pattern hierarchies or visitor patterns. In Haskell, the composition is a single line of code with no runtime overhead beyond the two independent summation passes.

B. Limitations

The primary limitation of the current system is the use of uniform, mode-specific emission factors that do not vary by vehicle age, occupancy, driving speed, or geographic context. Real-world emission factors exhibit substantial heterogeneity: a diesel bus in heavy urban congestion emits significantly more CO_2 per pkm than the same bus operating at free-flow speeds, and the grid-mix CO_2 factor for electric vehicles varies by country and even by time of day. The present model treats these heterogeneities as fixed parameters, which understates the variance in emission

estimates and may bias scenario results for cities with atypical vehicle fleets.

The dataset is limited to eight cities and two observation years, which restricts the statistical power of the correlation and trend analyses. With 16 observations, the Pearson correlation estimates carry wide confidence intervals, and the OLS trend slopes are based on only two data points per city, providing insufficient information to distinguish systematic trends from year-to-year variation. Expansion to a larger panel dataset with annual observations over a decade would substantially improve the reliability of both the correlation analysis and the trend projections.

The scenario normalisation procedure—which scales all modal shares proportionally after applying the target shifts—is a simplification of the behavioural mechanisms that govern how travellers redistribute across modes in response to policy interventions. In reality, travellers shift from car to transit along routes where transit is competitive on time and cost, not uniformly across all trip types. A more behaviourally accurate model would require a discrete choice framework parameterised by mode-specific utility functions, which is outside the scope of the current implementation.

C. Real-World Applicability

Despite these limitations, the system demonstrates clear utility as a rapid screening tool for urban transport policy. The predefined scenarios are calibrated to reflect the scale of modal share shifts that have been achieved in comparable cities through sustained policy investment: Amsterdam's cycling modal share increased by approximately 8 percentage points over a decade of infrastructure investment [11], and several Latin American cities have achieved 10–15 percentage point increases in bus rapid transit ridership within five years of dedicated lane implementation. The CO₂ reduction estimates produced by the Rail Push and Bus Rapid Transit scenarios (8.7–18.4% and 6.2–14.7% respectively) are therefore operationally meaningful rather than hypothetical.

The system's modular architecture also positions it well for integration with external data feeds. The DatasetLoader module accepts any CSV conforming to the defined thirteen-column schema, making it straightforward to ingest updated emission factor databases (e.g., the European Environment Agency's EMEP/EEA Air Pollutant Emission Inventory Guidebook) or city-specific modal share surveys without modifying the analytical modules. Similarly, the JSON export pipeline enables direct integration with urban dashboards, GIS platforms, and policy decision support systems.

VIII. CONCLUSION

This paper has presented a purely functional, higher-order Haskell system for modelling the sensitivity of urban transport emissions to changes in modal share and emission factor parameters. The system implements an activity-based emission model grounded in established transport emission accounting methodology, and exposes the emission computation as a higher-order abstraction that can be parameterised by any pollutant selector or perturbation operator without modification of the analytical framework. Simulation of five predefined policy scenarios across eight global cities demonstrates that car-to-rail modal shifts yield the largest CO₂ reductions (8.7–18.4%), that emission factor reductions through vehicle electrification have a higher marginal impact on total CO₂ than equivalent modal share shifts, and that cities with high baseline car dependency exhibit the greatest sensitivity to modal shift interventions.

From a computer science perspective, the key contribution of this work is the demonstration that the three-layer abstraction hierarchy—pure emission arithmetic, higher-order model combinators, and policy scenario simulation—provides a compositional, extensible, and verifiable framework for emission sensitivity analysis that is architecturally superior to equivalent imperative implementations. The use of Haskell's algebraic data types, type class instances, and higher-order functions as primary architectural mechanisms results in a codebase where each analytical concern is expressed in its most natural mathematical form, with no redundancy between the emission, sensitivity, and scenario computation paths.

IX. FUTURE WORK

Several directions for extending this work are identified. First, integration of a discrete choice behavioural model—specifically a multinomial logit model parameterised by mode-specific generalised cost functions—would replace the proportional normalisation heuristic with a behaviourally grounded redistribution mechanism. This extension is architecturally compatible with the current system, as the applyScenario function in PolicyModel can be replaced with a higher-order function that accepts a utility model without modifying the upstream emission computation.

Second, the emission factor data could be enriched with time-varying and speed-dependent values derived from the COPERT or MOVES emission modelling frameworks, enabling the system to account for congestion effects and fleet electrification trajectories. Third, the dataset could be expanded to include annual observations from a larger set of cities using open data portals such as the Global Urban Indicators Database or the C40 Cities Climate Finance Facility dataset, enabling robust panel econometric analysis

of the determinants of urban transport emissions across different city typologies.

Fourth, a property-based testing suite using Haskell's QuickCheck framework could be developed to verify key mathematical invariants of the emission model—such as the linearity of CO₂ in modal share, the conservation of total modal share after normalisation, and the monotonicity of CO₂ in car emission factor—providing machine-verified correctness guarantees for the core analytical functions. Fifth, the sensitivity analysis could be extended to second-order sensitivity indices (Sobol indices) using the functional composition capabilities of the Models module, enabling the system to quantify interaction effects between modal share parameters.

References

- [1] International Energy Agency, CO₂ Emissions from Fuel Combustion: Highlights, IEA, Paris, 2021.
- [2] Intergovernmental Panel on Climate Change, 2006 IPCC Guidelines for National Greenhouse Gas Inventories, Volume 2: Energy, IPCC, Geneva, 2006.
- [3] L. Ntziachristos and R. Boulter, EMEP/EEA Air Pollutant Emission Inventory Guidebook 2019: Road Transport, European Environment Agency, Copenhagen, 2019.
- [4] R. Cervero and C. Murakami, Rail and Property Development in Hong Kong: Experiences and Extensions, Urban Studies, vol. 46, no. 10, pp. 2019-2043, 2009.
- [5] J. R. Kenworthy and F. B. Laube, An International Sourcebook of Automobile Dependence in Cities, University Press of Colorado, Boulder, 1999.
- [6] International Energy Agency, Energy Technology Perspectives 2023: Pathways to Net-Zero, IEA, Paris, 2023.
- [7] S. Rao, S. Pachauri, F. Dentener, P. Kinney, Z. Klimont, K. Riahi, and W. Schoepp, Better Air for Better Health: Forging Synergies in Policies for Energy Access, Climate Change and Air Pollution, Global Environmental Change, vol. 23, no. 5, pp. 1122-1130, 2013.
- [8] P. Hudak, J. Peterson, and J. H. Fasel, A Gentle Introduction to Haskell, Version 98, Haskell.org, 1999.
- [9] K. Claessen and J. Hughes, QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs, in Proc. 5th ACM SIGPLAN International Conference on Functional Programming (ICFP), pp. 268-279, 2000.
- [10] J. Svenningsson and E. Axelsson, Combining Deep and Shallow Embedding for EDSL, in Trends in Functional Programming (TFP 2012), Lecture Notes in Computer Science, vol. 7829, Springer, pp. 21-36, 2013.
- [11] European Cyclists Federation, Cycling Statistics: Modal Share Trends in European Cities, ECF, Brussels, 2022.
- [12] A. de Nazelle et al., Improving Health Through Policies That Promote Active Travel: A Review of Evidence, Environment International, vol. 37, no. 4, pp. 766-777, 2011.